



Generación de Código

DISEÑO E IMPLEMENTACIÓN DE UN LENGUAJE

v0.1.0

1. Introducción

1

1. Introducción

De la sección anterior, es fácil ver que el cómputo de los tipos de datos se comporta de forma **sintetizada**, es decir, si el tipo de dato es un atributo de la gramática (*atribuida*), entonces dicho atributo se sintetiza desde las hojas del AST, y por lo tanto se obtiene de forma **ascendente**.

Sería posible utilizar la misma lógica para traducir un AST sintáctica y semánticamente válido, aprovechando el hecho de que la generación de código se puede definir de manera **local** sobre un nodo, como una función del generador de sus hijos.

En principio, generar el código implica traducir el programa de entrada en un lenguaje L_1 hacia un lenguaje L_2 . Para ello, es necesario recorrer el AST desde el nodo raíz, aplicando procedimientos de traducción sobre cada construcción encontrada (sobre cada nodo del árbol). Esta traducción puede involucrar una o varias “pasadas”, es decir, recorridos del AST, debido a que para ciertos lenguajes, el ordenamiento general de la estructura del programa de entrada podría no coincidir, o no ser traducible al ordenamiento del lenguaje de salida.

Para el siguiente programa ficticio:

$$123 + 4 * 89 - (x / 3)^8 \text{ with } x = 5$$

se desea obtener el siguiente programa de salida:

```
final int x = 5;
final double result = add(123, sub(mul(4, 89), exp(div(x, 3), 8)));
log(result);
```



```
        output(" = ")
        generate(p.INT)
        output("; \n")
        output("final double result = ")
        generate(p.expression)
        output("; \n")
        output("log(result); \n")
    }

    generate(expression e) {
        switch (e.expressionType) {
            case ADD_EXPRESSION:
                output("add(")
                generate(e.leftExpression)
                output(", ")
                generate(e.rigthExpression)
                output(")")
                break
            case SUB_EXPRESSION:
                output("sub(")
                generate(e.leftExpression)
                output(", ")
                generate(e.rigthExpression)
                output(")")
                break
            case MUL_EXPRESSION:
                ...
            case DIV_EXPRESSION:
                ...
            case EXP_EXPRESSION:
                ...
            case FACTOR_EXPRESSION:
                generate(e.factor)
                break
            default:
                log.error("Cannot generate unknown expression.")
        }
    }

    generate(factor f) {
        switch (f.factorType) {
            case ENCLOSED_FACTOR:
                generate(f.factor)
                break
            case ID_FACTOR:
                generate(f.ID)
                break
            case INT_FACTOR:
                generate(f.INT)
                break
        }
    }
}
```

```
        default:
            log.error("Cannot generate unknown factor.")
    }

    generate(ID i) {
        output(i.lexeme)
    }

    generate(INT i) {
        output(i.value)
    }
}
```

Algunos casos de generación, merecen una mención especial. Por ejemplo, al emitir la generación del nodo `program`, se tiene especial cuidado de emitir también espacios y fines de línea, de modo que el programa resultante sea válido sintácticamente.

Por otro lado, la generación **delega** la generación de subnodos a otros procedimientos, lo que permite simplificar la lógica al igual que lo hacía el script de *type-checking*.

En el caso de generar el código para las reglas de producción $expression \rightarrow factor$ y $factor \rightarrow OPEN\ factor\ CLOSE$, se realiza el **forwarding** de la generación, ya que no se emiten símbolos adicionales a ese nivel del árbol.

Al generar el código para $factor \rightarrow ID$, se aprovecha el hecho de que durante la generación del AST (anterior a la generación de código), ya se escaneó el nombre del ID utilizado en la sección **with** `x = 5` del programa de entrada, con lo cual, ya se sabe que ese ID debe ser `x`, y que el mismo está almacenado en dicho nodo por adelantado. Esto es importante, ya que si dicho ID no se encontrara disponible al momento de generar el código, entonces se debería ejecutar una fase anterior adicional para obtener el identificador, o de otro modo el programa de salida estaría incompleto, y por ende, sería inválido.

Finalmente, y al igual que para el chequeo de tipos, los nodos hoja del árbol emiten código en la salida, pero no ejecutan otra llamada al procedimiento `generate(.)`, ya que de otro modo no habría un caso base en esta recursión, y el generador nunca terminaría.

Como se observa en los procedimientos, principalmente para el nodo `program`, la salida mediante `output(.)` puede emitir código antes de descender por el árbol (si se encuentra antes de un llamado a `generate(.)`), o después (si se encuentra luego).